GPT-2 FROM SCRATCH

Building, Training and Deploying a Transformer-Based Language Model

---

Final Project Explanation Report

Submitted By	Aishani Billore
Institution	Swami Keshvanand Institute of Technology (SKIT)
Project Title	GPT-2 From Scratch — Language Model Training & Deployment
Domain	Deep Learning / Natural Language Processing (NLP)
Framework	PyTorch

This report explains, in clear and simple language, how a 50-million-parameter GPT-2 style language model was designed, implemented from scratch in PyTorch, trained on the WikiText-103 dataset, and deployed as an interactive Streamlit web application capable of next-word prediction. It documents the complete project workflow — dataset preparation, tokenization, model architecture, training methodology, evaluation approach, and the final application — along with a viva-style question and answer section for project defense.

Abstract

This project implements a smaller-scale version of OpenAI's GPT-2 architecture entirely from scratch, without relying on pre-built model APIs. The model — roughly 50 million parameters, with 8 transformer layers and 8 attention heads — was trained on WikiText-103, a large collection of high-quality Wikipedia text (~103 million tokens). Training used a cosine learning-rate schedule with warmup, the AdamW optimizer, gradient clipping, and weight tying between the input embedding and output projection layers. The trained model was deployed through a Streamlit web interface that accepts a user-typed prompt and predicts the most probable next word(s), demonstrating the core mechanism behind modern large language models. The project covers the complete pipeline: data preparation, tokenization, model design, training, validation, and deployment.

1. Project Overview

In one line: a 50-million-parameter GPT-2 language model was built from scratch in PyTorch, trained on the WikiText-103 dataset, and deployed as an interactive Streamlit web app that predicts the next word given a user's prompt.

In simpler terms, this project recreates — at a small, practical scale — the same core idea behind tools like ChatGPT: a model that reads a sentence and predicts what word is most likely to come next. Running that prediction step repeatedly is how full sentences and paragraphs get generated.

Aspect	Value
Model size	~50 Million parameters
Layers / Attention heads	8 / 8
Embedding size / Context length	512 / 512 tokens
Vocabulary size	50,257 tokens
Dataset	WikiText-103 (~103M tokens, Wikipedia)
Training steps	1,000 steps (~2M tokens seen)
Framework	PyTorch
Deployment	Streamlit web app (next-token prediction)

2. Tools and Technologies Used

Layer	Technology	Reason for Use
Language	Python 3.10+	Standard language for AI/ML development
Deep Learning	PyTorch	Flexible framework for building and training neural networks
Tokenizer	tiktoken (OpenAI)	Converts text to numbers, same method used by GPT-2
Dataset loading	HuggingFace datasets	Simple way to download WikiText-103
Progress bars	tqdm	Visual tracking of training progress
Numerics	NumPy	Efficient handling of large tokenized data files
UI	Streamlit	Rapidly build an interactive web application

3. Dataset — WikiText-103

WikiText-103 is a large collection of clean, well-written Wikipedia articles rated "Good" or "Featured" quality — essentially a well-organized textbook of formal English containing about 103 million tokens. It was chosen because it is free, high quality, and ideal for teaching a model correct grammar and factual writing style.

Dataset Statistics

Property	Value
Source	English Wikipedia
Total tokens	~103 million
Train split	~99.95% of data
Validation split	~0.05% of data
Train tokens	~117 million raw tokens
Validation tokens	~60,121 tokens
Train .bin file size	~224 MB
Validation .bin file size	~120 KB
Token storage type	uint16 (covers vocab of 50,257)
Format	Memory-mapped binary file (.bin)

Data Preparation Pipeline

- Download the raw Wikipedia articles from HuggingFace (load_dataset).
- Split into a training set (99.95%) and a small validation set (0.05%).
- Convert every article into numeric tokens using the GPT-2 BPE tokenizer.
- Write all tokens sequentially into one binary file, stored compactly as uint16.
- Use memory-mapping (np.memmap) so the program reads small chunks from disk instead of loading the entire 224 MB file into RAM at once — similar to reading one page at a time instead of memorizing the whole book first.

4. Tokenizer — Converting Text to Numbers

A neural network cannot read text directly — it only understands numbers. The tokenizer is the component that converts text into a sequence of numbers (tokens) and back again.

Byte-Pair Encoding (BPE)

- Starts by treating individual letters/bytes as the smallest units.
- Repeatedly merges the most frequently occurring pairs of characters into single units.
- Result: common words become a single token, while rare or unusual words are split into sub-word pieces.

Example

```
"Hello, world!" → [15496, 11, 995, 0]
```

```
"The apollo" → [464, 17261]
```

Tokenizer Statistics

Property	Value
Vocabulary size	50,257 tokens
BPE merge tokens	50,000
Raw byte tokens	256
Special token	1 (the <|endoftext|> marker)

5. Model Architecture — GPT-2

GPT-2 (Generative Pre-trained Transformer 2), originally developed by OpenAI in 2019, is an autoregressive language model — it examines a sequence of words and predicts the most likely next word, one step at a time.

Model Specification (This Project — ~50M Parameters)

Setting	Value	Description
num_layers	8	8 stacked transformer blocks
num_heads	8	8 parallel attention heads
embd_size	512	Each token represented as a 512-dimensional vector
context_length	512	Maximum tokens the model can see at once
vocab_size	50,257	Total number of possible tokens
Total parameters	~50 Million	Total trainable weights in the model

Rationale for 50M parameters: the original GPT-2 was released in sizes of 117M, 345M, 762M, and 1.5B parameters. A 50M-parameter version was chosen for this project since it trains substantially faster and runs on ordinary hardware, while still being large enough to demonstrate genuine transformer behaviour and learn meaningful language patterns.

6. End-to-End Workflow — From Text to Prediction

Example input: "The apollo mission was"

- 1. Tokenization: text is converted into numeric tokens, e.g. [464, 17261, 4365, 373] (4 tokens).
- 2. Token Embedding: each token is looked up in a 50,257 x 512 table, converting it into a 512-dimensional vector. Result: a 4 x 512 grid of numbers.
- 3. Position Embedding: positional information is added so the model knows word order. Still a 4 x 512 grid.
- 4. Token embedding and position embedding are summed to form the initial representation "x".
- 5. x passes through Block 1 of 8: LayerNorm -> Causal Self-Attention -> residual add -> LayerNorm -> MLP -> residual add.
- 6. This repeats through Blocks 2 to 8, progressively refining the representation.
- 7. A final LayerNorm is applied.
- 8. The LM Head (linear layer) projects the 512-dimensional output into a score for each of the 50,257 possible next tokens.
- 9. Softmax converts these scores into a probability distribution.
- 10. The model's top prediction is displayed, e.g. the token " a" with 3.26% probability.

7. Core Components of Each Transformer Block

A. Causal Self-Attention

Every token generates three vectors: a Query (what is this token looking for), a Key (what does this token represent), and a Value (the actual information carried). Tokens compare their Query against other tokens' Keys to determine how much attention to pay to each.

The term "causal" indicates that a token may only attend to itself and earlier tokens, never future ones — essential for autoregressive prediction, since the model must never have access to the answer it is trying to predict.

```
Attention(Q, K, V) = softmax(Q x K-transpose / sqrt(head_size)) x V
```

With 8 attention heads, each head operates on 512 / 8 = 64 dimensions and may specialize in different relationships (e.g. subject-tracking, verb-tracking). All 8 head outputs are concatenated back into 512 dimensions. Flash Attention (PyTorch's scaled_dot_product_attention) computes this identical mathematical operation more efficiently in terms of memory and speed.

Property	Value
Number of heads	8
Head size	512 / 8 = 64
Each head output shape	Batch x 8 x Tokens x 64
Combined output shape	Batch x Tokens x 512

B. MLP (Feed-Forward Network)

```
512 → Linear → 2048 → GELU → Linear → 512
```

Following attention, each token is independently passed through a small feed-forward network for further non-linear processing. GELU (Gaussian Error Linear Unit) is used as the activation function, consistent with the

original GPT-2 design. The hidden layer size is 4 times the embedding size (4 x 512 = 2048).

C. Layer Normalization

Applied before both the attention and MLP sub-layers (Pre-Norm configuration). It rescales each token's values to have mean 0 and standard deviation 1, which stabilizes and accelerates training.

D. Residual Connections

```
x = x + self.attn(self.ln_1(x)) # skip connection x = x + self.mlp(self.ln_2(x)) # skip connection
```

Rather than replacing the representation at each step, the block's output is added back to the original input. This creates an unobstructed path for gradients to flow backward through all 8 layers, preventing the vanishing gradient problem and enabling deeper networks to train effectively.

8. Weight Tying

```
self.transformer.wte.weight = self.lm_head.weight
```

The token embedding matrix (which converts tokens into vectors) and the LM head (which converts vectors back into token prediction scores) share the same underlying weight matrix. This design reduces the parameter count by approximately 26 million and improves performance, since the embedding and un-embedding operations are conceptually inverses of one another.

9. Weight Initialization

```
std = 0.02 if hasattr(module, 'SCALE_INIT'): std /= (2 * num_layers) ** 0.5
```

- Linear layers: initialized from a normal distribution with standard deviation 0.02.
- Residual projection layers (c_proj): scaled further by 1 / sqrt(2 x num_layers), reducing the output magnitude of residual branches so they do not dominate at the start of training.
- Embedding layers: also initialized from a normal distribution with standard deviation 0.02.

10. Training Methodology

Hyperparameter	Value	Purpose
Total steps	1,000	Constrained to fit within a limited training window (5-6 hours)
Batch size (B)	4	4 sequences processed per training step
Context length (T)	512	512 tokens per sequence
Tokens per step	2,048 (4 x 512)	
Total tokens seen	~2 Million (1000 x 2048)	
Max learning rate	3e-4	Peak update magnitude
Min learning rate	3e-5	Final learning rate (10% of max)
Warmup steps	50	Learning rate ramps up gradually over the first 50 steps
Weight decay	0.1	L2-style regularization to reduce overfitting
Optimizer	AdamW	Adam with decoupled weight decay
LR schedule	Cosine decay	Smooth decay in learning rate following warmup
Gradient clipping	1.0	Prevents exploding gradients

Learning Rate Schedule

```
Steps 0-50: Learning rate ramps linearly from 0 to 3e-4 (warmup) Steps 50-1000: Learning
rate decays smoothly (cosine curve) from 3e-4 to 3e-5
```

Loss Function

```
loss = -log(probability assigned to the correct next token)
```

Cross-entropy loss quantifies the discrepancy between the model's predicted probability distribution and the actual next token. A randomly initialized model would produce a loss of approximately ln(50257) = 10.8. A well-trained

model at this scale is expected to reach a loss below approximately 4.

Validation Procedure

Every 250 training steps, 10 batches from the held-out validation file (val.bin) are evaluated, and the resulting validation loss is logged to monitor generalization and detect overfitting.

11. Optimizer — AdamW

- Adam: adapts the effective learning rate per parameter using running estimates of the first and second moments of the gradients, converging substantially faster than plain SGD for transformer models.
- Decoupled weight decay (the "W" in AdamW): regularizes the model by penalizing large weights, helping to reduce overfitting.

Parameter Groups

- Decay group (parameter dimension >= 2): weight matrices and embeddings — weight decay applied.
- No-decay group (parameter dimension < 2): biases and LayerNorm parameters — no weight decay applied.

Beta coefficients: beta1 = 0.9 (momentum term), beta2 = 0.95 (second-moment term)

12. Data Loader Design

```
data = np.memmap("train.bin", dtype=np.uint16, mode='r')
```

The full tokenized dataset remains on disk; only the chunk required for the current batch is loaded into memory. This design choice is important because the 224 MB training file would otherwise place unnecessary strain on available RAM.

Batch Construction

```
[token_0, token_1, ..., token_512] -> x (input) [token_1, token_2, ..., token_513] -> y
(target)
```

Input and target sequences are offset by exactly one token, so the model learns to predict token t+1 given tokens 0 through t. After processing each batch of size B x T, the read pointer advances by B x T positions in the dataset.

13. Deployment — Streamlit Web Application

Application workflow, step by step:

- 1. The trained model checkpoint (logs/model_50M_wikitext.pt) is loaded into memory.
- 2. The user enters a text prompt, e.g. "The apollo mission was", into the web interface.
- 3. The prompt is tokenized using tiktoken.
- 4. A single forward pass of the tokens is run through the 50M-parameter model.
- 5. The logits at the final token position are extracted.
- 6. Softmax converts these logits into a probability distribution over the vocabulary.
- 7. The top-K most probable next tokens are displayed to the user along with their probabilities.

Scope and Limitation

The application performs single-step next-token prediction rather than full text generation. Generating complete sentences would require repeatedly predicting a token, appending it to the input, and re-running the model — a process known as autoregressive generation. The deployed interface demonstrates this foundational building block directly.

14. Evaluation — Loss, Accuracy, and Prediction Capability

Measurement Approach

Language models are typically evaluated using loss (and the related metric perplexity) rather than a simple classification-style accuracy percentage, since multiple different words can reasonably continue a given sentence. This project therefore reports cross-entropy loss as its primary evaluation metric.

Stage	Approx. Loss	Interpretation
Untrained / random model	~10.8 (ln 50,257)	Equivalent to pure random guessing over 50,257 tokens
Well-trained model (this scale)	Below ~4	Model reliably favours grammatically and contextually likely words

Given the constrained training budget (1,000 steps, ~2 million tokens seen) and the 50M-parameter scale, this project should be understood as an educational implementation and proof-of-concept rather than a production-grade language model. No single official accuracy percentage is reported in the project materials; performance is more accurately characterized by the achieved loss value and by direct qualitative inspection of the model's next-word predictions.

Capabilities Demonstrated

- Predicting a plausible next word for a given prompt (e.g. "The capital of France is" -> "Paris").
- Producing a ranked list of top-K likely next words with associated probabilities.
- Demonstrating learned grasp of English grammar and typical sentence structure.
- Capturing factual associations frequently present in the Wikipedia training corpus.
- Serving as the foundational component for full autoregressive text generation.

Known Limitations

- Cannot reliably produce long, coherent multi-sentence text without the repeated generation loop.
- Not guaranteed to be factually accurate — like all language models, it can produce confident but incorrect completions.
- Limited to patterns present in WikiText-103 (formal, encyclopedic English).
- Smaller in scale and less fluent than large production language models trained on far more data and compute.
- Restricted to a maximum context window of 512 tokens.

15. Key Project Parameters (Summary Table)

Metric	Value
Total parameters	~50 Million
Layers	8
Attention heads	8
Embedding dimension	512
Context window	512 tokens
Vocabulary size	50,257
Dataset	WikiText-103
Dataset size	~103 Million tokens
Training steps	1,000
Tokens trained on	~2 Million tokens
Training time	~2-3 hours (CPU)
Batch size	4 sequences
Tokens per batch	2,048
Max learning rate	3e-4
Optimizer	AdamW
Activation function	GELU
LR schedule	Cosine with warmup
Checkpoint size	~195 MB
Random-guess loss	~10.8
Well-trained loss target	Below ~4

16. Conclusion

This project successfully demonstrates a complete, end-to-end language model pipeline built entirely from first principles: dataset acquisition and preprocessing, tokenization, transformer architecture design, training with modern optimization techniques, and deployment through an interactive web application. It highlights a working understanding of the core mechanisms behind contemporary large language models — self-attention, positional encoding, residual connections, layer normalization, and autoregressive next-token prediction — implemented and trained without relying on pre-built model APIs.

Key Strengths of the Project

- Implemented entirely from scratch in pure PyTorch, without pre-built HuggingFace model APIs.
- Trained on a real, substantial dataset — 103 million tokens of clean Wikipedia text.
- Employs a proper training pipeline: cosine learning rate schedule, AdamW optimizer, gradient clipping, and weight tying.
- Uses Flash Attention (via PyTorch's scaled_dot_product_attention) for computational efficiency.
- Uses memory-efficient data loading (np.memmap) to handle a dataset larger than convenient RAM usage.
- Delivers a functional, production-style user interface built with Streamlit.
- Incorporates basic MLOps practices: checkpoint saving, validation evaluation, and training metric logging.
